Untitled-1

```cpp
class ThreadPool {
public:
    explicit ThreadPool(size_t thread_count = std::thread::hardware_concurrency()) {
        for (size_t i = 0; i < thread_count; ++i) {
            // C++20: jthread 会自动将 std::stop_token 传入线程函数
            workers.emplace_back([this](std::stop_token stoken) {
                while (true) {
                    std::function<void()> task;
                    {// 如果收到停止请求且队列为空，wait 返回 false；否则返回 true
                        std::unique_lock<std::mutex> lock(queue_mutex);
                        bool has_task = condition.wait(lock, stoken, [this] {
                            return !tasks.empty();
                        });
                        if (!has_task) return;
                        task = std::move(tasks.front());
                        tasks.pop();
                    }
                    task();
                }
            });
        }
    }
    ~ThreadPool() {// C++20: 主动向所有 worker 发出停止请求
        for (auto& worker : workers)
            worker.request_stop();
    }
    template<typename F, typename... Args>
    auto enqueue(F&& f, Args&&... args) {// 推导函数执行后的返回值类型
        using return_type = std::invoke_result_t<F, Args...>;
        // C++20: Lambda 初始化捕获中直接展开参数包
        auto task = std::make_shared<std::packaged_task<return_type()>>(
            [func = std::forward<F>(f), ...args = std::forward<Args>(args)]() mutable {
                return std::invoke(func, args...);
            }
        );
        std::future<return_type> res = task->get_future();
        {// 将任务包装成无参数无返回值的 std::function 存入队列
            std::unique_lock<std::mutex> lock(queue_mutex);
            tasks.emplace([task]() { (*task)(); });
        }
        condition.notify_one();
        return res;
    }
private:
    std::vector<std::jthread> workers;          // C++20 新线程类
    std::queue<std::function<void()>> tasks;    // 任务队列
    std::mutex queue_mutex;                     // 互斥锁
    std::condition_variable_any condition;      // 必须使用 any 以配合 stop_token
};
```